



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**FinanceFlow: Plataforma Web de Gestión
Financiera e Inteligencia Artificial para el
Control de Gastos Personales**

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Sebastian Arce Bracamonte

(2019062886)

Brant Antony Chata Choque

(2020067577)

Tacna – Perú

2026

FinanceFlow: Plataforma Web de Gestión
Financiera e Inteligencia Artificial para el
Control de Gastos Personales
Diccionario de Datos

Versión 1.0

ÍNDICE GENERAL

1. Modelo Entidad / relación	4
1.1. Diseño lógico	4
1.2. Diseño Físico	4
2. DICCIONARIO DE DATOS	4
2.1. Tablas	4
1.1. Procedimientos Almacenados	4
1.2. Lenguaje de Definición de Datos (DDL)	5
1.3. Lenguaje de Manipulación de Datos (DML)	5

Diccionario de Datos

1. Modelo Entidad / relación

1.1. Diseño lógico

Entidades principales:

- Usuario: Almacena la información y credenciales de los usuarios del sistema.
- Movimiento: Representa las transacciones financieras (ingresos y egresos) asociadas a un usuario.
- Log: Mantiene un registro de auditoría de las acciones realizadas por los usuarios en el sistema.

Relaciones:

- Un Usuario puede tener muchos Movimientos (1:N).
- Un Usuario puede tener muchos Logs (1:N).
- Un Movimiento puede estar asociado a muchos Logs (1:N) de forma opcional (para rastrear cambios en una transacción específica).

1.2. Diseño Físico

El diseño físico está implementado en **MongoDB** (Base de datos NoSQL orientada a documentos). Cada "Entidad" corresponde a una "Colección" y los registros son "Documentos" BSON. Las relaciones se manejan mediante el uso de referencias (`ObjectId`).

2. DICCIONARIO DE DATOS

2.1. Tablas

Tabla 1: Usuarios (Usuario)

Nombre de la Tabla: `usuarios`

Descripción de la Tabla: Almacena la información de las cuentas de usuario y sus credenciales de autenticación.

Objetivo: Permitir el acceso seguro al sistema y asociar la información de movimientos a cuentas específicas.

Relaciones con otras Tablas:

- Relación 1:N con `movimientos` (Un usuario tiene muchos movimientos).
- Relación 1:N con `logs` (Un usuario genera muchos registros de auditoría).

Descripción de los campos:

Nro .	Nombre del campo	Tipo dato longitud	Permite nulos	Clave primaria	Clave foránea	Descripción del campo
1	_id	ObjectId	No	Sí	No	Identificador único del usuario (Generado por MongoDB).
2	nombre	String	No	No	No	Nombre completo del usuario.
3	email	String	No	No	No	Correo electrónico único del usuario.
4	passwordHash	String	No	No	No	Contraseña encriptada para autenticación.
5	creadoEn	Date	Sí	No	No	Fecha y hora de creación de la cuenta (Por defecto actual).
6	actualizadoEn	Date	Sí	No	No	Fecha y hora de la última actualización del registro.
7	estado	String	Sí	No	No	Estado de la cuenta ('activo' o 'inactivo').
8	resetPasswordToken	String	Sí	No	No	Token para recuperación de contraseña.

9	resetPasswordExpires	Date	Sí	No	No	Fecha de expiración del token de recuperación.
---	----------------------	------	----	----	----	--

Tabla 2: Movimientos (Movimiento)

Nombre de la Tabla: movimientos

Descripción de la Tabla: Guarda las transacciones financieras (ingresos o egresos) realizadas.

Objetivo: Mantener el seguimiento del balance financiero registrando entradas y salidas de dinero.

Relaciones con otras Tablas:

- Relación N:1 con `usuarios` (Muchos movimientos pertenecen a un usuario).
- Relación 1:N con `logs` (Un movimiento puede tener un historial de modificaciones).

Descripción de los campos:

Nr o.	Nombre del campo	Tipo dato longitud	Permitenulos	Clave primaria	Clave foránea	Descripción del campo
1	<code>_id</code>	ObjectId	No	Sí	No	Identificador único del movimiento (Generado por MongoDB).
2	<code>tipo</code>	String	No	No	No	Tipo de transacción ('ingreso' o 'egreso').
3	<code>nombre</code>	String	No	No	No	Nombre descriptivo de la transacción.
4	<code>monto</code>	Number	No	No	No	Valor monetario de la transacción (Mínimo 0).
5	<code>categoria</code>	String	No	No	No	Categoría asignada (ej. Salario, Alimentación).
6	<code>userId</code>	ObjectId	No	No	Sí	Referencia al <code>_id</code> de la tabla

						usuarios (Dueño del movimiento).
7	fecha	Date	No	No	No	Fecha en la que ocurrió el movimiento.
8	estado	String	Sí	No	No	Estado de la transacción ('activo' o 'inactivo').
9	creadoEn	Date	Sí	No	No	Fecha y hora de registro.
10	actualizadoEn	Date	Sí	No	No	Fecha y hora de la última modificación.

Tabla 3: Logs (Log)

Nombre de la Tabla: logs

Descripción de la Tabla: Archiva el rastro de auditoría de acciones y eventos del sistema.

Objetivo: Mantener un historial seguro de "quién hizo qué y cuándo" para fines de seguridad y monitoreo.

Relaciones con otras Tablas:

- Relación N:1 con usuarios (El usuario que ejecutó la acción).
- Relación N:1 con movimientos (El movimiento afectado opcionalmente).

Descripción de los campos:

Nr o.	Nombre del campo	Tipo dato longitud	Permite nulos	Clave primaria	Clave foránea	Descripción del campo
1	_id	ObjectId	No	Sí	No	Identificador único del log (Generado por MongoDB).
2	accion	String	No	No	No	Acción o evento realizado (ej. "create", "delete", "login").
3	usuarioId	ObjectId	No	No	Sí	Referencia al _id de la tabla usuarios (Quien realizó la acción).

4	movimientoId	ObjectId	Sí	No	Sí	Referencia al _id de movimientos asociado a la acción (Opcional).
5	fecha	Date	Sí	No	No	Fecha y hora del evento.
6	descripcion	String	Sí	No	No	Contexto o resumen legible de la acción.

1.1. Procedimientos Almacenados

Procedimiento 1: Crear Usuario

Nombre de del SP (Servicio): `usuarios.service.js / crearUsuario`

Descripción del procedimiento: Hashea la contraseña proporcionada y registra un nuevo usuario en la base de datos.

Objetivo: Facilitar el registro seguro de nuevos clientes.

Relaciones con otros procedimientos: Dispara la creación de un registro en `logs.service.js`.

Descripción de los campos:

Nro .	Nombre del campo	Tipo dato	IN / OUT	Descripción del campo
1	nombre	String	IN	Nombre del nuevo usuario.
2	email	String	IN	Correo electrónico.
3	password	String	IN	Contraseña en texto plano (será encriptada).
4	usuarioObjeto	Object	OUT	Datos del usuario creado (excluyendo el hash de la contraseña).

Procedimiento 2: Registrar Movimiento

Nombre de del SP (Servicio): `movimientos.service.js / crearMovimiento`

Descripción del procedimiento: Crea un nuevo documento en la colección de movimientos asociado a un usuario concreto.

Objetivo: Permitir el registro de un ingreso o gasto.

Relaciones con otros procedimientos: Llama a `logs.service.js` para asentar el alta.

Descripción de los campos:

Nr o.	Nombre del campo	Tipo dato	IN / OUT	Descripción del campo
1	userId	ObjectId	IN	ID del usuario autenticado.
2	movimientoData	Object	IN	Objeto conteniendo: tipo, nombre, monto, categoria, fecha.
3	nuevoMovimiento	Object	OUT	El registro completo del movimiento almacenado.

1.2. Lenguaje de Definición de Datos (DDL)

SQL

```
// Definición de Usuario (Mongoose Schema)
const UsuarioSchema = new mongoose.Schema({
  nombre: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  passwordHash: { type: String, required: true },
  estado: { type: String, default: 'activo' }
});

// Definición de Movimiento (Mongoose Schema)
const MovimientoSchema = new mongoose.Schema({
  tipo: { type: String, required: true, enum: ['ingreso', 'egreso'] },
  nombre: { type: String, required: true },
  monto: { type: Number, required: true },
  categoria: { type: String, required: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true }
});
```

1.3. Lenguaje de Manipulación de Datos (DML)

INSERT:

SQL

```
// Crear movimiento
db.movimientos.insertOne({
  tipo: "ingreso",
  nombre: "Salario Mensual",
  monto: 1500,
  categoria: "Sueldo",
  userId: ObjectId("..."),
  fecha: new Date()
});
```

UPDATE:

SQL

```
// Actualizar estado de usuario
db.usuarios.updateOne(
  { _id: ObjectId("...") },
  { $set: { estado: "inactivo", actualizadoEn: new
Date() } }
);
```

DELETE:

SQL

```
// Eliminar lógicamente un movimiento (soft-delete)
db.movimientos.updateOne(
  { _id: ObjectId("...") },
  { $set: { estado: "inactivo" } }
);
```

1.4. Lenguaje de Control de Datos (DCL)

SQL

```
// Creación de rol en la Base de Datos
db.createRole({
  role: "appRole",
  privileges: [
    { resource: { db: "sistema_balance", collection:
"" }, actions: [ "find", "insert", "update" ] }
  ],
  roles: []
});
```

```
});

// Asignar el rol al usuario de la aplicación
db.createUser({
  user: "appUser",
  pwd: "securePassword123",
  roles: [ { role: "appRole", db: "sistema_balance" } ]
});
```

1.5. Lenguaje de Control de Transacciones (TCL)

SQL

```
const session = await mongoose.startSession();
session.startTransaction();
try {
  const movimiento = await Movimiento.create([ { /*
datos */ }, { session } ]);
  await Log.create([ { accion: 'create', movimientoId:
movimiento[0]._id }, { session } ]);

  // Commit de transaccion (COMMIT)
  await session.commitTransaction();
} catch (error) {
  // Rollback si ocurre un error (ROLLBACK)
  await session.abortTransaction();
  throw error;
} finally {
  session.endSession();
}
```

1.6. Lenguaje de Consulta de Datos (DQL)

SELECT simple:

SQL

```
// Encontrar todos los movimientos de un usuario específico
db.movimientos.find({
  userId: ObjectId("..."),
  estado: "activo"
}).sort({ fecha: -1 });
```

SELECT avanzado (Agrupación / SUM):

SQL

```
// Calcular el total de ingresos y egresos por usuario  
db.movimientos.aggregate([  
  { $match: { userId: ObjectId("..."), estado: "activo" }  
},  
  { $group: {  
    _id: "$tipo",  
    total: { $sum: "$monto" }  
  } }  
]);
```